

Entity Framework Core Inheritance (TPH, TPT, and TPC)

In this article, I will discuss **Entity Framework Core (EF Core) Inheritance (TPH, TPT, and TPC)** with Examples. Please read our previous article discussing [Stored Procedures in Entity Framework Core](#) with Examples.

What do you mean by Entity Framework Core Inheritance?

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (known as a derived class or child class) to inherit properties and methods from another class (known as a base class or parent class). This promotes code reusability and the creation of more specialized classes from general ones.

In Entity Framework Core (EF Core), inheritance refers to how an object-oriented domain model's inheritance relationships are represented in a relational database. EF Core supports several inheritance mapping strategies, allowing us to model entities that follow an OOP paradigm in a relational database. The three main inheritance mapping strategies in EF Core are:

- **Table Per Hierarchy (TPH):** All entities in the inheritance hierarchy are stored in a single table.
- **Table Per Type (TPT):** Each entity type in the inheritance hierarchy is stored in its own table.
- **Table Per Concrete Class (TPC):** Each concrete (non-abstract) entity class has its own table, with no table for abstract base classes.

Table Per Hierarchy (TPH) Inheritance in Entity Framework Core

Table Per Hierarchy (TPH), or single-table inheritance, stores all entities in the inheritance hierarchy in a single database table. A discriminator column is used to distinguish between entity types in the hierarchy.

- **Single Table:** One table stores all entities in the hierarchy.
- **Discriminator Column:** A special column identifies the specific type of each row.
- **Efficient Querying:** Since only one table is involved, queries are generally faster but may include many null columns for properties not applicable to some entities.

Example to Understand Table Per Hierarchy (TPH) in EF Core

Let's follow a step-by-step example of implementing Table Per Hierarchy (TPH) in EF Core.

Base Class:

We need to start by defining a base class representing the common properties for all the entities in our inheritance hierarchy. So, create the following Base Entity. This base class contains common properties for all derived entities.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class BaseEntity
    {
        public int Id { get; set; }
        public string CommonProperty { get; set; }
    }
}
```

Derived Classes:

Next, we need to create derived classes that inherit from the base class and add specific properties. Each derived class can have additional properties specific to that entity type. So, create the following Derived Entities:

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class DerivedEntityA : BaseEntity
    {
        public string PropertyA { get; set; }
    }
    public class DerivedEntityB : BaseEntity
    {
        public string PropertyB { get; set; }
    }
}
```

Each derived class adds properties unique to that entity type. For example, DerivedEntityA has PropertyA, and DerivedEntityB has PropertyB.

Configuring TPH with EF Core Fluent API:

In the DbContext class, we need to use Fluent API to configure the discriminator column and specify how EF Core should handle the hierarchy. So, modify the EFCoreDbContext class as follows:

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<BaseEntity>()
                .ToTable("Entities")
                .HasDiscriminator<string>("entity_type")
                .HasValue<DerivedEntityA>("EntityA")
                .HasValue<DerivedEntityB>("EntityB");
        }
        public DbSet<BaseEntity> BaseEntities { get; set; }
    }
}
```

Code Explanation:

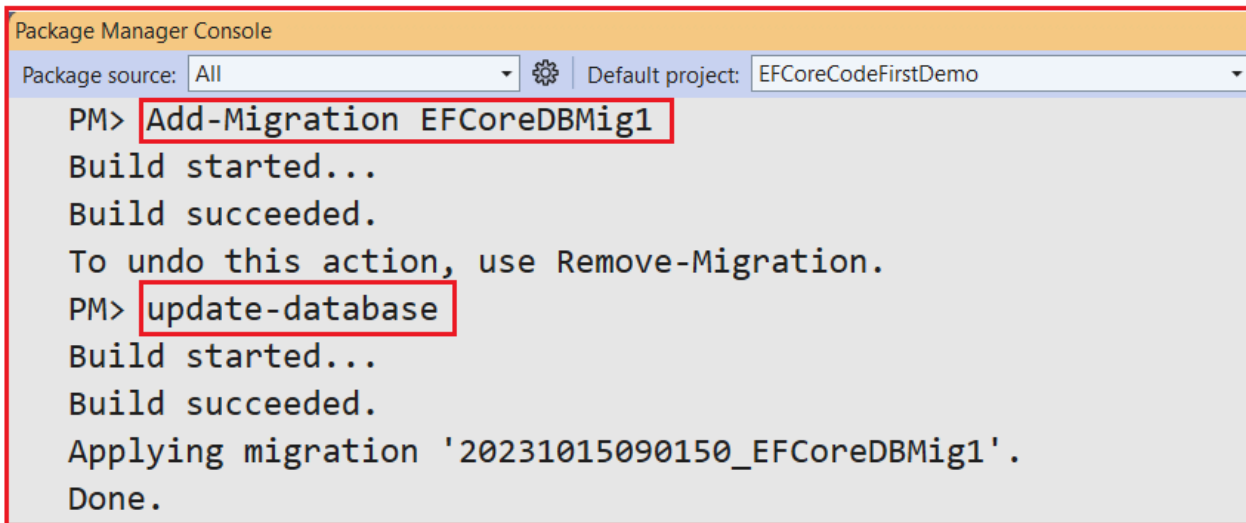
EF Core will add a discriminator column to the “Entities” table named “entity_type”. This column will store a value that indicates the type of entity each row represents. The value for the discriminator column will be either “EntityA” or

“EntityB”. EntityA represents the DerivedEntityA, and EntityB represents the DerivedEntityB.

When we query the database, EF Core automatically uses the discriminator column to determine the entity type and return the appropriate derived class instances. When we insert or update entities, EF Core sets the discriminator value appropriately based on the type of entity we are working with.

Generate Migration and Update Database:

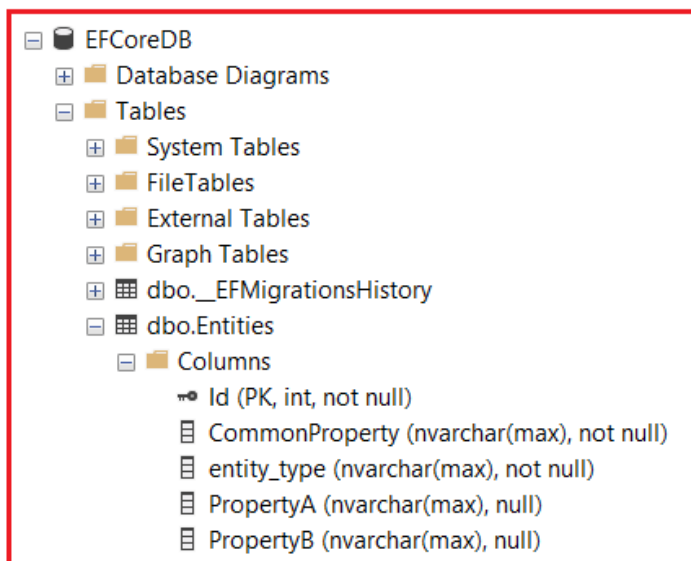
Open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands as follows.

A screenshot of the Package Manager Console in Visual Studio. The console shows the following commands and output:

```
PM> Add-Migration EFCoreDBMig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20231015090150_EFCoreDBMig1'.
Done.
```

The commands and their immediate outputs are highlighted with red boxes in the original image.

Now, if you verify the database, then you will see the following:

A screenshot of the SQL Server Enterprise Manager showing the database schema for EFCoreDB. The tree view is expanded to show the 'dbo.Entities' table. The columns listed are:

- Id (PK, int, not null)
- CommonProperty (nvarchar(max), not null)
- entity_type (nvarchar(max), not null)
- PropertyA (nvarchar(max), null)
- PropertyB (nvarchar(max), null)

The entire screenshot is enclosed in a red border.

Test TPH Inheritance Implementation

Now, modify the Program class to check whether TPH Inheritance is working as expected with Entity Framework core. The following example code is self-explained, so please read the comment lines for a better understanding:

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
```

```

public class Program
{
    static async Task Main(string[] args)
    {
        try
        {
            //Use Inheritance in our Code
            //Now, you can create and work with instances of the derived
classes
            //and save them to the database
            using (var context = new EFCoreDbContext())
            {
                var derivedEntityA = new DerivedEntityA { PropertyA =
"SomeValueA", CommonProperty = "SomeCommonValue" };
                var derivedEntityB = new DerivedEntityB { PropertyB =
"SomeValueB", CommonProperty = "SomeCommonValue" };
                //context.BaseEntites.Add(derivedEntityA);
                //context.BaseEntites.Add(derivedEntityB);
                //List<BaseEntity> entities = new List<BaseEntity>();
                //entities.Add(derivedEntityA);
                //entities.Add(derivedEntityB);
                //context.BaseEntites.AddRange(entities);
                context.BaseEntites.AddRange(derivedEntityA,
derivedEntityB);

                context.SaveChanges();
                Console.WriteLine("Entities are Added");
            }
            //Query the Inheritance Hierarchy
            //You can query the inheritance hierarchy using LINQ queries
            using (var context = new EFCoreDbContext())
            {
                var baseEntities = context.BaseEntites.ToList();
                foreach (var entity in baseEntities)
                {
                    if (entity is DerivedEntityA derivedEntityA)
                    {
                        Console.WriteLine($"\\tDerivedEntityA: Id:
{derivedEntityA.Id}, PropertyA: {derivedEntityA.PropertyA}, CommonProperty:
{derivedEntityA.CommonProperty}");
                    }
                    else if (entity is DerivedEntityB derivedEntityB)
                    {
                        Console.WriteLine($"\\tDerivedEntityB: Id:
{derivedEntityB.Id}, PropertyA: {derivedEntityB.PropertyB}, CommonProperty:
{derivedEntityB.CommonProperty}");
                    }
                }
            }
            Console.Read();
        }
        catch (Exception ex)

```

```

    {
        Console.WriteLine($"Error: {ex.Message}"); ;
    }
}
}

```

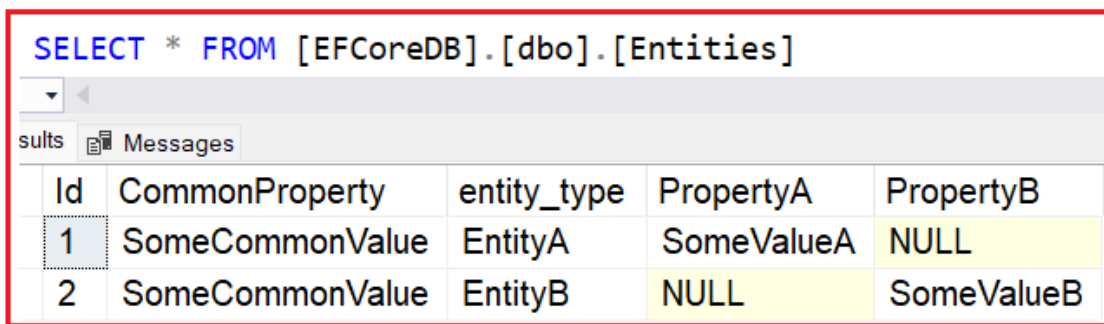
Output:

```

Entities are Added
    DerivedEntityA: Id: 1, PropertyA: SomeValueA, CommonProperty: SomeCommonValue
    DerivedEntityB: Id: 2, PropertyA: SomeValueB, CommonProperty: SomeCommonValue

```

If you verify the database table, you will see the following data.



The screenshot shows a SQL query window with the following query: `SELECT * FROM [EFCoreDB].[dbo].[Entities]`. Below the query, there is a tab labeled 'Messages' and a table of results. The table has five columns: Id, CommonProperty, entity_type, PropertyA, and PropertyB. There are two rows of data. The first row has Id: 1, CommonProperty: SomeCommonValue, entity_type: EntityA, PropertyA: SomeValueA, and PropertyB: NULL. The second row has Id: 2, CommonProperty: SomeCommonValue, entity_type: EntityB, PropertyA: NULL, and PropertyB: SomeValueB.

Id	CommonProperty	entity_type	PropertyA	PropertyB
1	SomeCommonValue	EntityA	SomeValueA	NULL
2	SomeCommonValue	EntityB	NULL	SomeValueB

Points to Consider with TPH

- **Performance:** Queries are fast since no joins are needed, but tables can grow large due to nullable columns.
- **Schema Flexibility:** Not ideal if derived types have vastly different sets of properties.
- **Discriminator Column:** Automatically added to differentiate entity types.

So, the above example demonstrates how to implement Table-Per-Hierarchy (TPH) inheritance in EF Core to store related entities in a single database table while preserving the type information. In our upcoming articles, we will discuss [Real-time Examples of Table-Per-Hierarchy \(TPH\) inheritance in EF Core](#).

Table Per Type (TPT) Inheritance in Entity Framework Core

Table Per Type (TPT) stores each class in the hierarchy in a separate table. The base class properties are stored in one table, while each derived class has its own table containing only its specific properties. In the Table Per Type (TPT) strategy:

- **Separate Tables:** Each class in the hierarchy is mapped to its own table.
- **Foreign Key Relationships:** Derived tables have foreign keys referencing the base table.
- **No Nullable Columns:** Each table contains only relevant columns.
- **Normalized Data:** Reduces redundancy but requires joins for querying.

Example to Understand Table Per Type (TPT) in EF Core:

Let's walk through a step-by-step example of implementing Table Per Type (TPT) in Entity Framework Core.

Define Base Class:

Decorate the base class with the [Table] attribute to specify the table name. This is the parent class in our model. It will be represented by its own table in the database. So, create the following BaseEntity class.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    [Table("BaseTable")]
    public class BaseEntity
    {
        [Key]
        public int Id { get; set; }
        public string CommonProperty { get; set; }
    }
}
```

Define Derived Classes:

Decorate each derived class with the [Table] attribute. Each derived class will be represented by its own table in the database. The table for a derived class includes a primary key column that is also a foreign key referencing the base class table. The following are our derived classes:

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    [Table("DerivedTable1")]
    public class DerivedEntity1 : BaseEntity
    {
        public string Property1 { get; set; }
    }
    [Table("DerivedTable2")]
    public class DerivedEntity2 : BaseEntity
    {
        public string Property2 { get; set; }
    }
}
```

Configuring TPT with Fluent API:

In your DbContext class, use the modelBuilder to configure the TPT inheritance strategy using the HasBaseType method. So, modify the EFCoreDbContext class as follows:

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
```

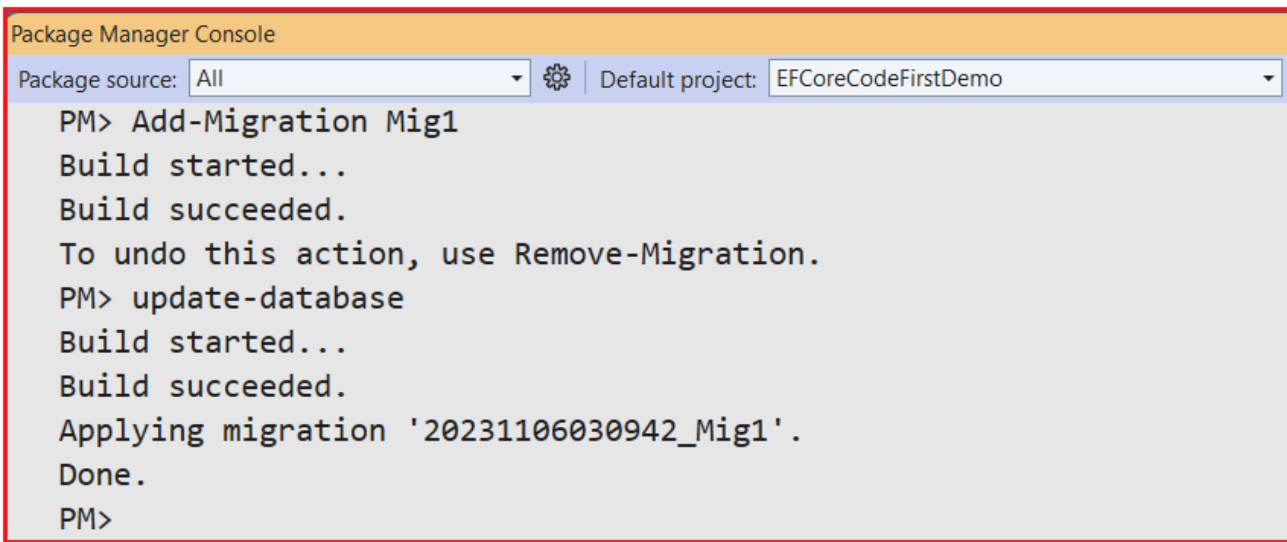
```

{
    //Configuring the Connection String
    optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServ
}
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<DerivedEntity1>().HasBaseType<BaseEntity>();
    modelBuilder.Entity<DerivedEntity2>().HasBaseType<BaseEntity>();
}
public DbSet<BaseEntity> BaseEntites { get; set; }
}
}

```

Generate Migration and Update Database:

Open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands as follows.

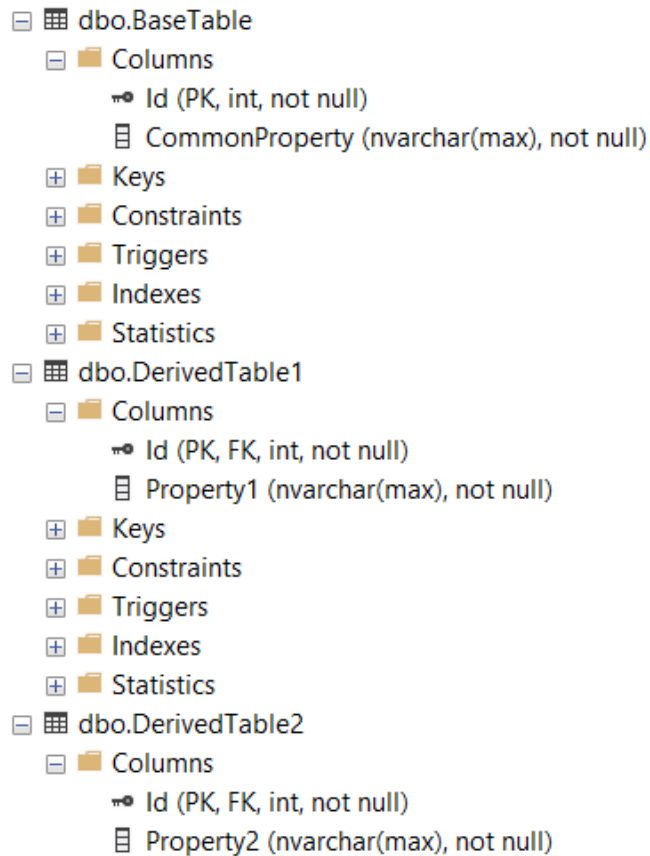


```

Package Manager Console
Package source: All [v] [gear] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20231106030942_Mig1'.
Done.
PM>

```

Now, if you verify the database, then you will see the following:



Test TPT Inheritance Implementation:

Next, modify the Program class to check whether TPT Inheritance is working as expected with Entity Framework core. The following example code is self-explained, so please read the comment lines for a better understanding:

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                //Use Inheritance in our Code
                //Now, you can create and work with instances of the derived
classes
                //and save them to the database
                using (var context = new EFCoreDbContext())
                {
                    var derivedEntityA = new DerivedEntity1 { Property1 =
"SomeValue1", CommonProperty = "SomeCommonValue" };
                    var derivedEntityB = new DerivedEntity2 { Property2 =
"SomeValue2", CommonProperty = "SomeCommonValue" };
                    context.BaseEntites.AddRange(derivedEntityA,
derivedEntityB);
                }
            }
        }
    }
}

```

```

        context.SaveChanges();
        Console.WriteLine("Entities are Added");
    }
    //Query the Inheritance Hierarchy
    //You can query the inheritance hierarchy using LINQ queries
    using (var context = new EFCoreDbContext())
    {
        var baseEntities = context.BaseEntites.ToList();
        foreach (var vehicle in baseEntities)
        {
            if (vehicle is DerivedEntity1 derivedEntityA)
            {
                Console.WriteLine($"\\tDerivedEntityA: Id:
{derivedEntityA.Id}, Property1: {derivedEntityA.Property1}, CommonProperty:
{derivedEntityA.CommonProperty}");
            }
            else if (vehicle is DerivedEntity2 derivedEntityB)
            {
                Console.WriteLine($"\\tDerivedEntityB: Id:
{derivedEntityB.Id}, Property2: {derivedEntityB.Property2}, CommonProperty:
{derivedEntityB.CommonProperty}");
            }
        }
        Console.Read();
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}"); ;
    }
}
}
}
}
}

```

Output:

```

Entities are Added
    DerivedEntityA: Id: 1, Property1: SomeValue1, CommonProperty: SomeCommonValue
    DerivedEntityB: Id: 2, Property2: SomeValue2, CommonProperty: SomeCommonValue

```

If you verify the database table, you will see the following data.

```

SELECT * FROM BaseTable;
SELECT * FROM DerivedTable1;
SELECT * From DerivedTable2;

```

120 %

Results Messages

	Id	CommonProperty
1	1	SomeCommonValue
2	2	SomeCommonValue

	Id	Property1
1	1	SomeValue1

	Id	Property2
1	2	SomeValue2

Points to Consider with TPT Inheritance in EF Core

- **Performance:** TPT involves joins when querying, which can lead to performance overhead.
- **Data Integrity:** Better integrity due to normalized structure and use of foreign keys.
- **Schema Complexity:** Adds complexity with multiple tables but avoids nullable columns.

So, the above example demonstrates how to implement Table-Per-Type (TPT) inheritance in EF Core, where each type has its own database table. In our upcoming articles, we will discuss [Real-time Examples of Table-Per-Type \(TPT\) inheritance in EF Core](#).

Table Per Concrete Type (TPC) Inheritance in Entity Framework Core:

Table Per Concrete Class (TPC) maps each concrete class (non-abstract class) to its own table, but there is no table for the base abstract class. Each table contains all properties, including those inherited from the base abstract class. In the Table Per Concrete Class (TPC) strategy:

- **Separate Tables:** Each concrete class has its own table.
- **Duplicate Columns:** Inherited properties are duplicated in each table.
- **Efficient Querying:** No joins are required since each table contains all properties.

Example to Understand Table Per Type (TPT) in EF Core:

Let's walk through a step-by-step example of implementing Table Per Concrete Type (TPC) in Entity Framework Core.

Define Base Abstract Class:

Start by defining a base class (abstract class or interface) that represents the common properties that need to be shared by all the child classes in the inheritance hierarchy. In our example, we will use the following abstract class.

```
using System.ComponentModel.DataAnnotations;
namespace EFCoreCodeFirstDemo.Entities
{
    public abstract class BaseEntity
    {
        [Key]
        public int Id { get; set; }
        public string CommonProperty { get; set; }
    }
}
```

Define Derived Classes:

Next, create the derived classes that inherit from the abstract base class. Each derived class should have its own set of properties. So, create the following derived concrete classes.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class DerivedEntity1 : BaseEntity
    {
        public string Property1 { get; set; }
    }
    public class DerivedEntity2 : BaseEntity
    {
        public string Property2 { get; set; }
    }
}
```

Configuring TPC with Fluent API

In our DbContext class, we need to use the modelBuilder to configure the TPC inheritance strategy. All the concrete types are mapped to individual tables in the TPC mapping pattern. For TPC mapping call the **modelBuilder.Entity<BaseEntity>().UseTpcMappingStrategy()** on the base entity type. So, please modify the EFCoreDbContext class as shown below.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
        }
    }
}
```

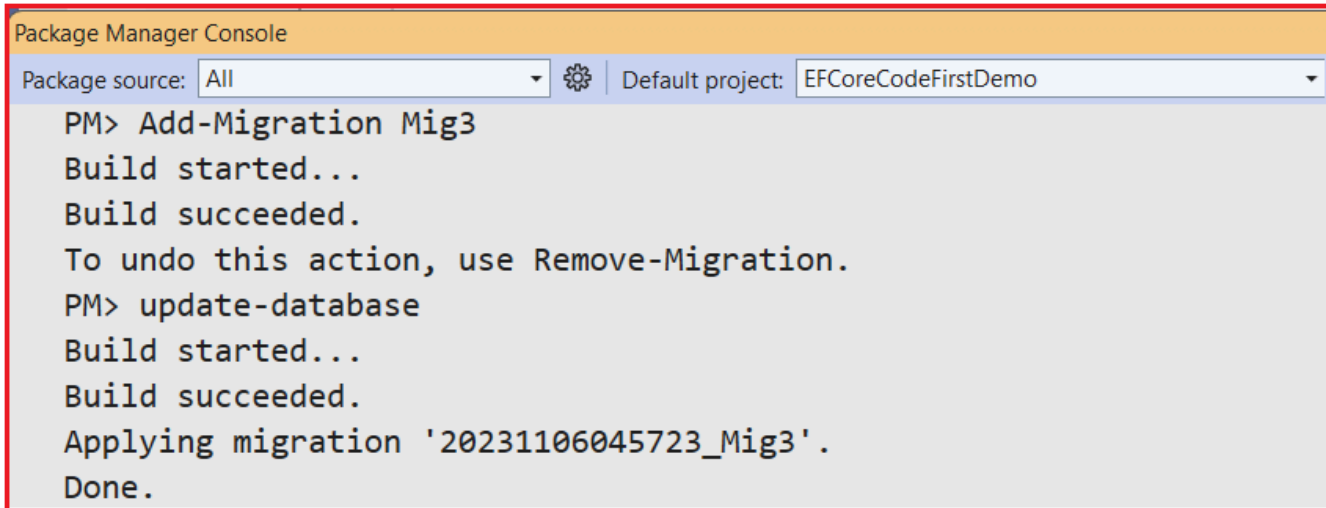
```

        modelBuilder.Entity<BaseEntity>().UseTpcMappingStrategy();
        modelBuilder.Entity<DerivedEntity1>().ToTable("DerivedTable1");
        modelBuilder.Entity<DerivedEntity2>().ToTable("DerivedTable2");
    }
    public DbSet<BaseEntity> BaseEntites { get; set; }
}
}

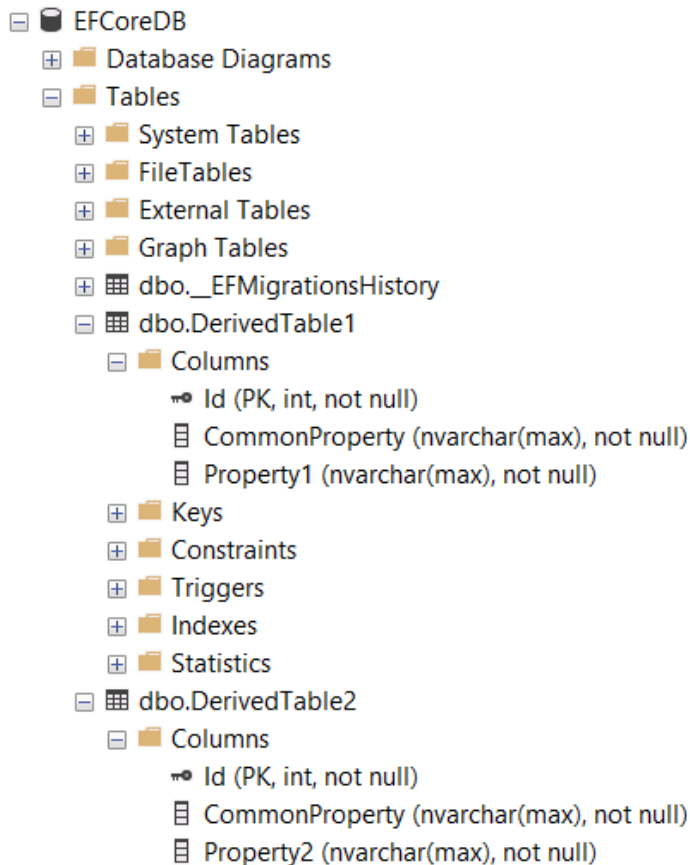
```

Generate Migration and Update Database:

Open the Package Manager Console and Execute the Add-Migration and Update-Database commands as follows.



Once you execute the above commands, verify the database. You should see separate database tables for each derived type (DerivedTable1 and DerivedTable2), as shown in the below image. These tables will include both the properties inherited from the base class (BaseEntity) and the properties specific to each derived class.



Test TPC Inheritance Implementation:

Next, modify the Program class to check whether TPC Inheritance is working as expected with Entity Framework core. The following example code is self-explained, so please read the comment lines for a better understanding:

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                //Use Inheritance in our Code
                //Now, you can create and work with instances of the derived
                classes
                //and save them to the database
                using (var context = new EFCoreDbContext())
                {
                    var derivedEntityA = new DerivedEntity1 { Property1 =
                    "SomeValue1", CommonProperty = "SomeCommonValue" };
                    context.BaseEntites.Add(derivedEntityA);
                    var derivedEntityB = new DerivedEntity2 { Property2 =
                    "SomeValue2", CommonProperty = "SomeCommonValue" };
                    context.BaseEntites.Add(derivedEntityB);
                }
            }
        }
    }
}
```

```

        context.SaveChanges();
        Console.WriteLine("Entities are Added");
    }
    //Query the Inheritance Hierarchy
    //You can query the inheritance hierarchy using LINQ queries
    using (var context = new EFCoreDbContext())
    {
        var DerivedEntities1 =
context.BaseEntites.OfType<DerivedEntity1>().ToList();
        foreach (DerivedEntity1 derivedEntity1 in
DerivedEntities1)
        {
            Console.WriteLine($"\\tDerivedEntityA: Id:
{derivedEntity1.Id}, Property1: {derivedEntity1.Property1}, CommonProperty:
{derivedEntity1.CommonProperty}");
        }
        var DerivedEntities2 =
context.BaseEntites.OfType<DerivedEntity2>().ToList();
        foreach (DerivedEntity2 derivedEntity2 in
DerivedEntities2)
        {
            Console.WriteLine($"\\tDerivedEntityA: Id:
{derivedEntity2.Id}, Property2: {derivedEntity2.Property2}, CommonProperty:
{derivedEntity2.CommonProperty}");
        }
    }
    Console.Read();
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}"); ;
}
}
}
}

```

In the above example, the OfType method filters the entities retrieved from the DbSet based on their concrete type, either DerivedEntity1 or DerivedEntity2. The filtered entities are then loaded into memory using the ToList method and displayed in the console, as shown in the image below.

```

Entities are Added
    DerivedEntityA: Id: 1, Property1: SomeValue1, CommonProperty: SomeCommonValue
    DerivedEntityA: Id: 2, Property2: SomeValue2, CommonProperty: SomeCommonValue

```

Now, if you verify the database table, you will see the following data:

<pre>SELECT * FROM DerivedTable1; SELECT * From DerivedTable2;</pre>		
% Results Messages		
Id	CommonProperty	Property1
1	SomeCommonValue	SomeValue1
Id	CommonProperty	Property2
2	SomeCommonValue	SomeValue2

So, the above example demonstrates how to implement Table Per Concrete Type (TPC) inheritance in EF Core. In our upcoming articles, we will discuss [Real-time Examples of Table-Per-Concrete Type \(TPC\) inheritance in EF Core](#).

Comparing TPH, TPT, and TPC in Entity Framework Core:

TPH is the fastest as it requires no joins, while TPT can suffer from slower queries due to joins, and TPC has no joins but may result in redundant data. TPH has a simpler schema but may lead to many nullable columns. TPT normalizes data but adds complexity. TPC avoids joins but increases redundancy. For a detailed comparison between TPH, TPT, and TPC in Entity Framework Core, please have a look at the following diagram:

Strategy	Description	Advantages	Disadvantages
TPH	Single table for entire hierarchy with discriminator column.	- Simple schema - Better performance due to no joins - Easy to implement	- Wide tables with many nullable columns - Not suitable for hierarchies with vastly different properties
TPT	Separate table for each class, linked via foreign keys.	- Normalized schema - No nullable columns - Clear separation of data	- Joins required in queries - Potential performance impact - More complex schema
TPC	Separate table for each concrete class, no base table.	- Efficient querying of specific types - No joins required - No nullable columns	- Data redundancy - Increased storage - Complex schema - Difficult to enforce data integrity

Choosing the Right Strategy Between TPH, TPT, and TPC:

Choosing the right strategy depends on your application's needs. If query performance and simplicity are key, TPH might be the best fit. However, if you want a clean separation of data, TPT or TPC may be more appropriate. So, consider the following when choosing an inheritance mapping strategy:

- **Performance:** TPH generally offers better performance due to simpler queries.
- **Schema Complexity:** TPH results in a simpler schema, while TPT and TPC can increase complexity.

- **Data Integrity:** TPT provides better data integrity due to normalized tables.
- **Data Redundancy:** TPC may lead to data redundancy and increased storage requirements.

In Short,

- TPH is useful for shared properties and a simple schema.
- TPT is ideal when you want a normalized database with fewer nullable columns.
- TPC works well when there are highly distinct entities with little shared data.

So, Entity Framework Core provides flexible options for mapping inheritance hierarchies to relational databases. Understanding TPH, TPT, and TPC strategies allows you to choose the most suitable approach for your application's requirements, balancing performance, complexity, and data integrity.

In the next article, I will discuss [Table Per Hierarchy Inheritance in Entity Framework Core](#) with Examples. In this article, I try to explain **Entity Framework Core (EF Core) Inheritance** with Examples. I hope you enjoy this Entity Framework Core (EF Core) Inheritance article.